Chapter 5: Whitebox testing

Reference

This chapter refers from the book:

Mastering Software Quality Assurance: Best Practices, Tools and Techniques for Software Developers

Introduction to Software Testing

Outline

5.1. Whitebox testing introduction

5.2. Whitebox testing technique

- 5.2.1. Control flow testing technique
- 5.2.2. Data flow testing technique

5.3. Automation unit test

5.4. Excercise

Outline

5.1. Whitebox testing introduction

5.2. Whitebox testing technique

- 5.2.1. Control flow testing technique
- 5.2.2. Data flow testing technique

5.3. Automation unit test

5.4. Excercise

2. White-box Testing (WBT) introduction

- WBT relies on a specific algorithm, on the internal data structure of the module to be tested, to determine if the module is performing correctly.
- Therefore, WBT tester must have skills and knowledge to be able to understand in detail about the code to be tested.
- WBT usually takes a lot of time and effort
- For important modules, which perform the main computation of the system, this approach is necessary.
- White box testing methods:
  - Control flow testing
  - Data flow testing

Outline

5.1. Whitebox testing introduction

5.2. Whitebox testing technique

- **5.2.1. Control flow testing technique**
- 5.2.2. Data flow testing technique

5.3. Automation unit test

5.4. Excercise

5.2.1. Control flow testing technique

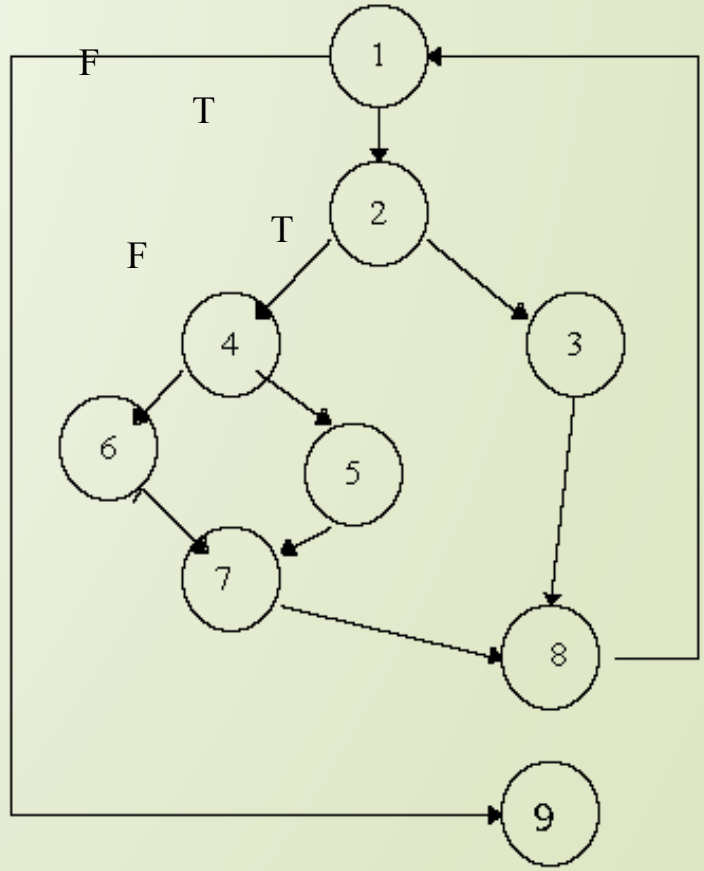
Some definitions

- **Execution path**: is a corresponding software unit execution script: an ordered list of instructions to be executed for a particular run of the software unit. It starts from the entry point of the software unit and stop at the end of the software unit.
- **Goal of control flow testing**: is to ensure that all execution paths of the software unit under test run correctly. Unfortunately, in reality, the effort and time to achieve the above goal is very large, even on small software units.

Example

```
1:      WHILE NOT EOF LOOP
2:         Read Record;
2:         IF field1 equals 0 THEN
3:            Add field1 to Total
3:            Increment Counter
4:         ELSE
4:             IF field2 equals 0 THEN
5:                 Print Total, Counter
5:                 Reset Counter
6:             ELSE
6:                 Subtract field2 from Total
7:             END IF
8:         END IF
8:         Print "End Record"
9:      END LOOP
9:      Print Counter
```

Example of execution paths :
1, 9
1, 2, 3, 8, 1, 9
1, 2, 4, 5, 7, 8, 1, 9
1, 2, 4, 6, 7, 8, 1, 9



Discussion (1)

- For code:

```
for (i=1; i<=1000; i++)
    for (j=1; j<=1000; j++)
        for (k=1; k<=1000; k++)
            doSomethingWith(i,j,k);
```

  There is only 1 execution path, but length is 1000*1000*1000 = 1 billion statements `doSomethingWith(i,j,k)`.

- For code:

```
if (c1) s11 else s12;
if (c2) s21 else s22;
if (c3) s31 else s32;

...
    if (c32) s321 else s322;
```

  There are 2^{32} = 4 billion execution paths.

Discussion(2)

- Some execution paths maybe miss :

```
if (a>0) doIsGreater();
if (a==0) dolsEqual();
// missing case a < 0 - if (a<0) dolsLess();
```

- A execution path is tested to be pass with a test case. However, it is still incorrect with another test cases:

```
int blech (int a, int b) { return a/b; }
```

When testing, we select b <> 0 => the result is correct. However, when b = 0 the function will be error.

Testing coverage

- From discussion above, we should test the minimum number of test cases that result in maximum confidence. But how to determine the minimum number of test cases that can return results with maximum confidence?
- **Coverage:** is the ratio of components actually tested to the population after testing the selected test cases. The larger the coverage, the higher the reliability.
  - The component involved can be a command, decision point, subcondition, execution path, or a combination thereof.

Coverage level 0 & 1

- Level 0 : freely test. The users will test the remain parts. This is the level of testing that is not really responsible
- Level 1 : each statement must be excuted one time. E.g.,
  - For function foo, 2 following test cases will obtain 100% coverage level 1 :
    - TC1. foo(0,0,0,0), return 0
    - TC2. foo(1,1,1,1), return 1
    - However, they cannot find error divide for 0 at line 8.

```
1. float foo(int a, int b, int c, int
   d) {
2. return e;
3. float e;
4. if (a==0)
5.     return 0;
6. int x = 0;
7. if ((a==b) || ((c==d) && bug(a)))
8.     x = 1;
9. e = 1/x;
10.}
```

Coverage level 2

- Each decision must be excuted >=1 time for true case, and >=1 time for false case. It's called Branch coverage.

Line	Predicate	True	False
3	(a == 0)	Test Case 1 foo(0, 0, 0, 0) return 0	Test Case 2 foo(1, 1, 1, 1) return 1
6	((a==b) OR ((c == d) AND bug(a)))	Test Case 2 foo(1, 1, 1, 1) return 1	Test Case 3 foo(1, 2, 1, 2) return 1

- With TC1, TC2 from previou slide, we obain 3/4 x 75% branch coverage. Add test case 3 :
- TC3. foo(1,2,1,2), we obain 100% branch coverage.

Coverage level 3

- Each subcondition must be excuted >= 1 time for true case, >=1 time for false case. We call it's subcondition coverage.

Predicate	True	False
a==0	Test Case 1 foo(0, 0, 0, 0) return 0	Test Case 2 foo(1, 1, 1, 1) return value 0
a==b	Test Case 2 foo(1, 1, 1, 1) return 1	Test Case 3 foo(1, 2, 1, 2) division by zero!
c==d		Test Case 3 foo(1, 2, 1, 2) division by zero!
bug(a)		

Coverage level 4

- Each subcondition of each decision must be excuted >=1 time for true case, >=1 time for false case, and this true (false case) must effect the result of decision.
- We call it's branch & subcondition coverage.

Basis Path Testing (by Tom McCabe)

Basis Path Testing (by Tom McCabe)

- Guideline:
- From testing module, create control flow graph G.
- Compute complexity Cyclomatic of G (called C).
  - V(G) = E - N + 2, for E is number of edges, N is number of nodes.
  - V(G) = P + 1, for P is number of decision node.
  - Note: if V(G) >10, we should devide the module to be submodules to reduce probability of error
- Create C linear basic paths to be tested.
- Create test case for each basic path.

Basis Path Testing (cont.)

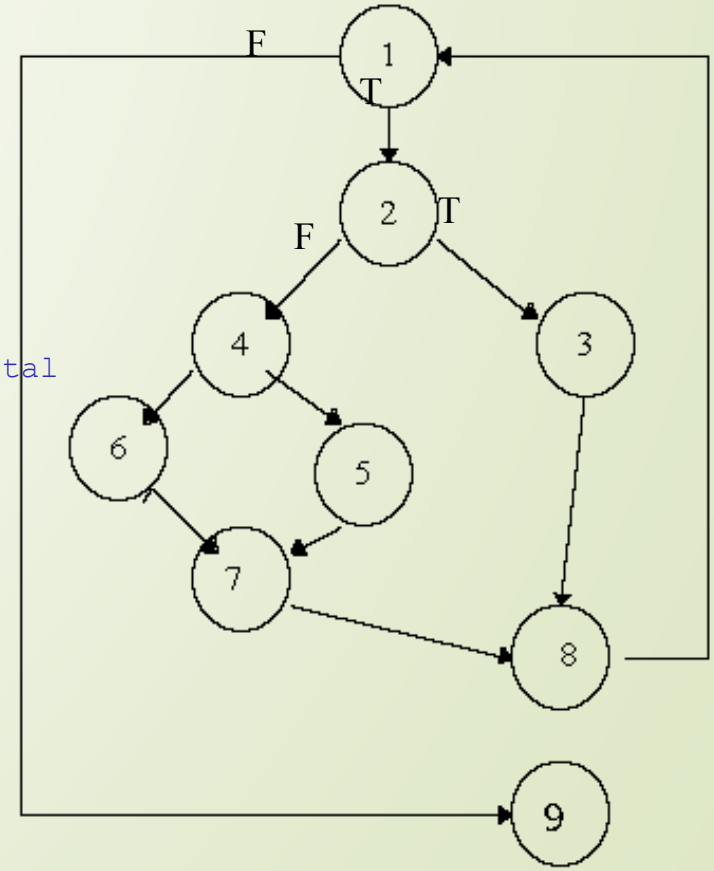
Create C linear basic paths to be tested

1. Create first basic path. It should be the most popular path
2. For second path, change the edge for the first decision node and try to keep maximum of first basic path.
3. For 3rd path, change the edge of second decision node and try to keep maximum of first basic path.
4. Continue with path 4th, 5th… until all decison node are consider (enough C path).

Example

```
1:   WHILE NOT EOF LOOP
2:      Read Record;
2:      IF field1 equals 0 THEN
3:            Add field1 to Total
3:            Increment Counter
4:         ELSE
4:             IF field2 equals 0 THEN
5:                 Print Total, Counter
5:                 Reset Counter
6:             ELSE
6:                 Subtract field2 from Total
7:             END IF
8:         END IF
8:         Print "End Record"
9:      END LOOP
9:      Print Counter
```

- We have C = 3 + 1 = 4, path:
- 1, 9
- 1, 2, 3, 8, 1, 9
- 1, 2, 4, 5, 7, 8, 1, 9
- 1, 2, 4, 6, 7, 8, 1, 9

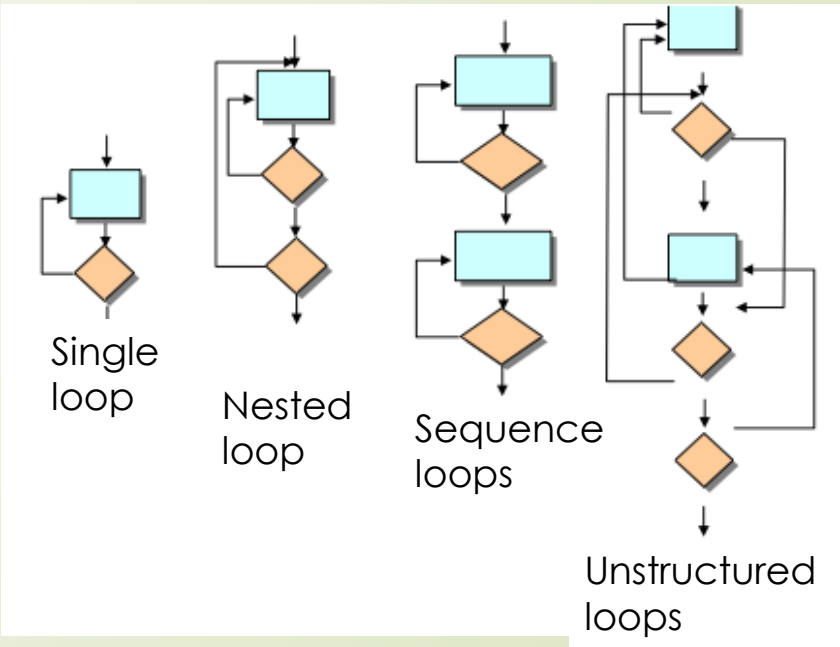


Loop testing

Loop testing

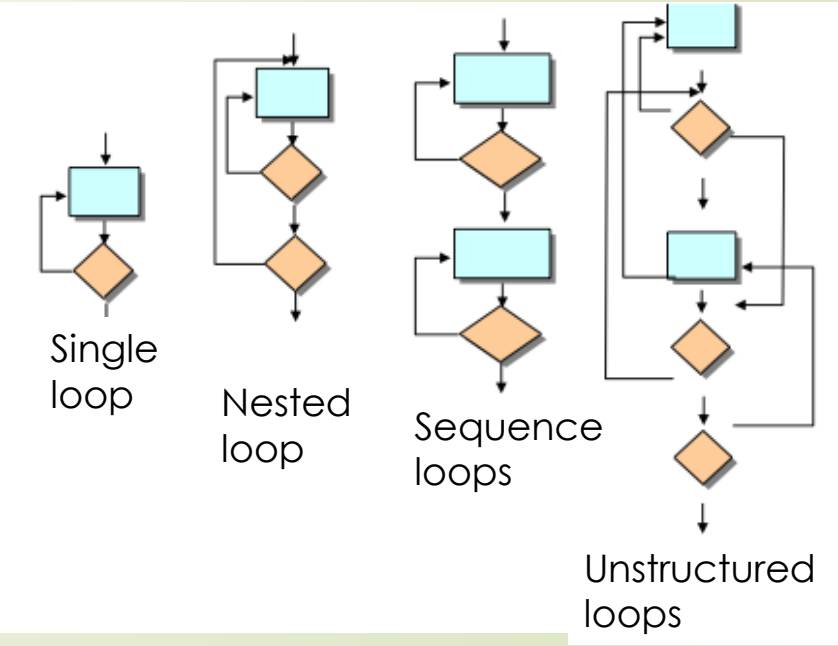
- Focus on checking the validity of the loop structure.
- For single loop:
  - Skip loop
  - loop 1 time
  - loop k times (k <n)
  - loop n, n + 1 times

With n is the maximum number of iterations



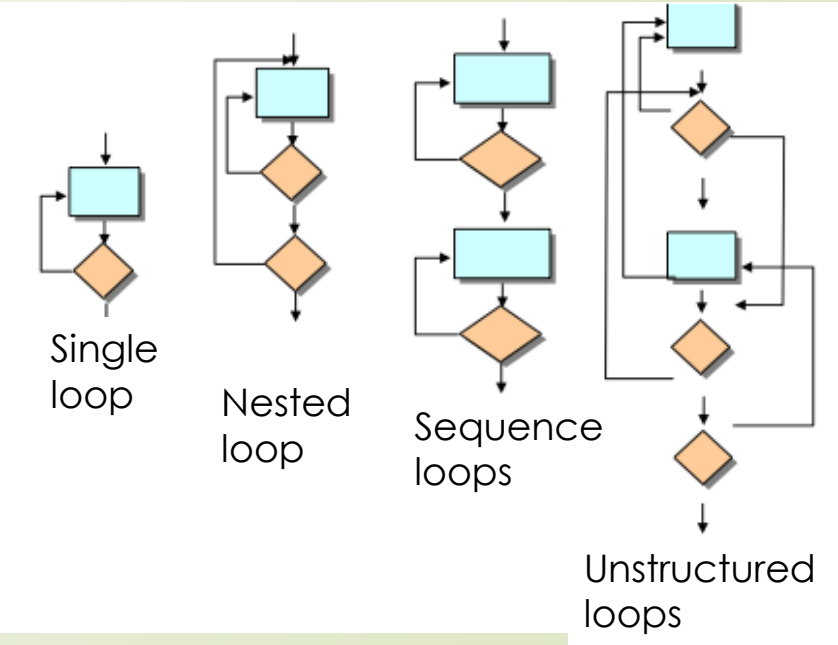
Loop testing (cont.)

- For nested loop:
  - Start with the innermost loop. Set the iteration parameters for the outer loops to the minimum value
  - Check the min+1 parameter, a typical value, max -1, max for the inner loop while the iteration parameters of the outer loops are minimal
  - Continue with outer loops until all loops are tested



Loop testing (cont.)

- For sequence loops:
  - Do similar to nested loop
- For unstructured loops?



Loop testing: Example

```
1. // LOOP TESTING EXAMPLE PROGRAM import java.io.*;
2. class LoopTestExampleApp {
3. // ------------------ FIELDS ----------------------
4. public static BufferedReader keyboardInput = new BufferedReader(new
   InputStreamReader(System.in));
5.      private static final int MINIMUM = 1;
6.      private static final int MAXIMUM = 10;
7. // ------------------ METHODS --------------------
8. /* Main method */
9. public static void main(String[] args) throws IOException {
10.     System.out.println("Input an integer value:");
11.     int input = new Integer(keyboardInput.readLine()).intValue();
12.     int numberOfIterations=0;
13.     for(int index=input;index >= MINIMUM && index <= MAXIMUM;index++)
14.     {
15.             numberOfIterations++;
16.     } // Output and end
17.     System.out.println("Number of iterations = " + numberOfIterations); }
18.}
```

Loop testing: Example (cont.)

Input	Result
11	0 (skip loop)
10	1 (loop 1 time)
5	5 (loop k time)
1	10 (loop n time)
0	0 (skip loop)

Outline

5.1. Whitebox testing introduction

5.2. Whitebox testing technique

- 5.2.1. Control flow testing technique
- **5.2.2. Data flow testing technique**

5.3. Automation unit test

5.4. Excercise

Data flow testing

- is an effective method for identifying issues with variables:
  - The statement of assigning or entering data into a variable is incorrect.
  - Missing variable definition before use
  - The axiom is wrong (due to the wrong execution of the execution flow)…
  - Each variable should have a good life cycle through a 3-step sequence: created, used, and deleted.
  - Only commands within the scope of the variable can access/process the variable.
    - Scope: global, local

Data flow testing (cont.)

Analysis of life cycle of a variable

- Commands access a variable through one of the following three actions:
  - d (define): defines a variable, assigns a specified value to the variable (entering data into a variable is also an operation of assigning a value to the variable).
  - r(reference) : refer to the value of the variable (usually through an expression).
  - u(undefine) : cancel (delete) the variable.
- Thus, if the symbol ~ is to describe the state in which the variable does not exist yet, we have the first three possibilities for processing on a variable:
  - ~d : variable does not exist and is defined with the specified value.
  - ~r : variable does not exist and can be used immediately (what value?)
  - ~u : variable does not exist and is destroyed (unusual).

Data flow testing (cont.)

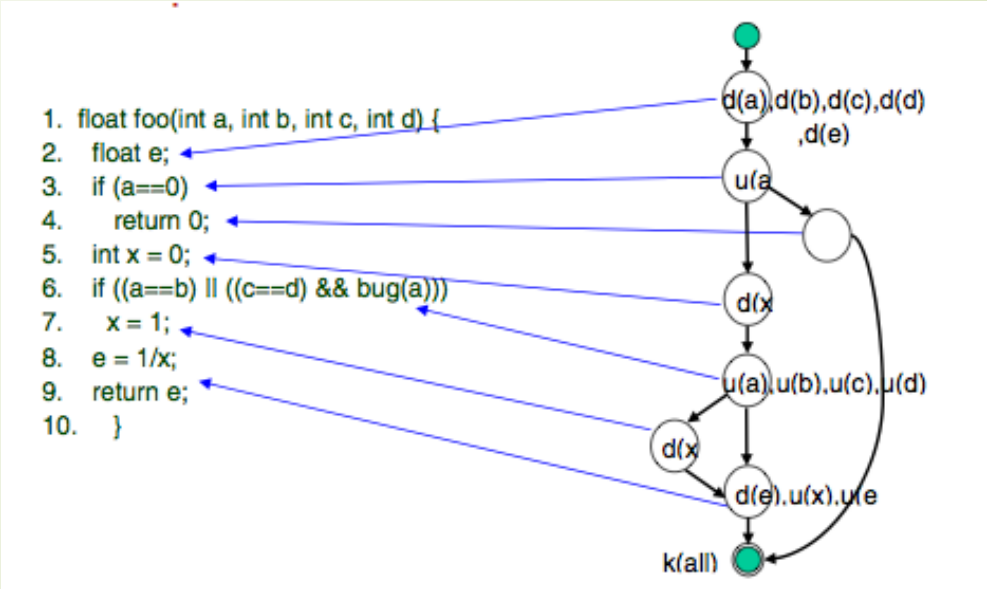
Analysis of life cycle of a variable (cont)

- 3 different variable operations combined to create 9 pairs:
  - dd : variable is defined and then defined : *a bit strange, maybe correct and acceptable, but can also have programming errors.*
  - dr : variable defined then used : *correct and normal sequence.*
  - du : variable defined and then deleted : *a bit strange, maybe correct and acceptable, but can also have programming errors.*
  - rd : variable is used then define a new value : *reasonable*.
  - rr : variable is used and then used : *reasonable*.
  - ru : variable used and then destroyed : *reasonable*.
  - ud : variable deleted then redefined : *acceptable*.
  - ur : variable deleted and then used : *error*.
  - uu : the variable is deleted and then deleted again : *probably a programming error*.

Data flow testing (cont.)

Data flow grahp

- To describing different life scenarios of variables.
- Creating data flow graph is similarly to creating control flow graph of the module under test + labeling variable action .



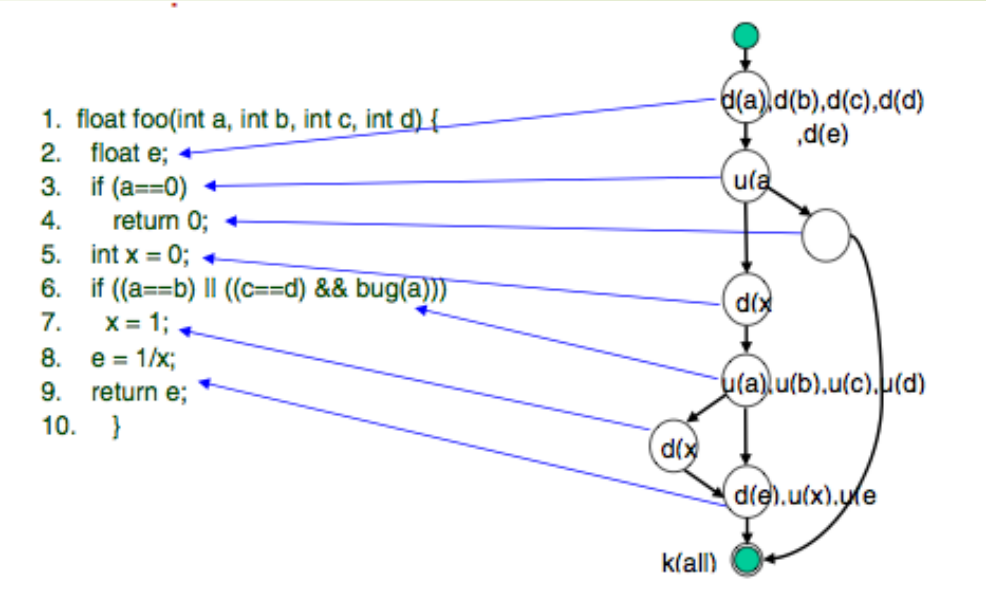
Data flow testing (cont.)

Data flow testing process

- Create control flow graph, then change to data flow graph.
- Compute Cyclomatic of graph (C = P +1).
- Create C basic paths.
- Testing live cycle for each data variable:
  - Each variable has maximum C scenario for variable action.
  - For each scenario, finding abnormal pair actions.

Data flow testing: Example.

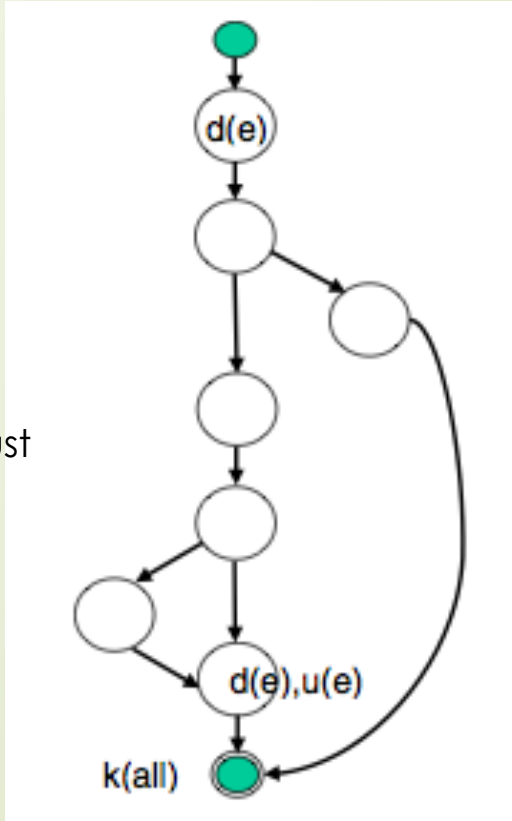
- Graph has 2 decision node, then C = 2 +1 = 3.
- Function has 4 input parameter, 2 local variables: a, b, c, d, e, x.



Data flow testing: Example.

- Scenario 1 : ~dduk
- Scenario 2: ~dduk (like scenario 1).
- Scenario 3: ~dk

Scenario 1 & 2 have abnormal pair dd. So, we must determine whether it is related to an error or not.



Outline

5.1. Whitebox testing introduction

5.2. Whitebox testing technique

- 5.2.1. Control flow testing technique
- 5.2.2. Data flow testing technique

5.3. Automation unit test

5.4. Excercise

Tool

- http://pathcrawler-online.com:8080
- PathCrawler's principal functionality, and the one demonstrated in this online version, is to automate structural unit testing by generating test inputs for full coverage of the C function under test. Full coverage can mean all feasible execution paths or *k-path* coverage which restricts the all-path criterion to paths with at most *k* consecutive loop iterations. These are available in this online version. PathCrawler can also be used to satisfy other coverage criteria (such as branch coverage, MC-DC,...), to generate supplementary tests to improve coverage of an existing functional test suite or to generate just the tests necessary to cover the part of the code which has been impacted by a modification.

Outline

5.1. Whitebox testing introduction

5.2. Whitebox testing technique

- 5.2.1. Control flow testing technique
- 5.2.2. Data flow testing technique

5.3. Automation unit test

5.4. Excercise

Excercise

- Execute tool http://pathcrawler-online.com:8080 and find:
  - coverage results
  - Test cases
  - Unit code corresponding with test cases
- Study unit test with Junit.